

Autonomous Surface Vessels Path Following and Static Obstacle Collision Avoidance Using Deep Reinforcement Learning

Nam H. Tran

Australian Maritime College
University of Tasmania
Launceston, Australia
hoangnam.tran@utas.edu.au

Hung D. Nguyen

Australian Maritime College
University of Tasmania
Launceston, Australia
h.d.nguyen@utas.edu.au

Peter King

Autonomous Maritime Systems
Laboratory
University of Tasmania
Launceston, Australia
p.d.king@utas.edu.au

Minh Tran

School of Science, Engineering
and Technology
RMIT University
Ho Chi Minh City, Vietnam
minh.tranquang@rmit.edu.vn

Abstract—Autonomous Surface Vessels (ASVs) require reliable path following and collision avoidance for safe maritime operations, particularly in cluttered or uncertain environments. This paper proposes a Deep Reinforcement Learning (DRL) framework that integrates a 3-DOF dynamic ship model with a simulated LiDAR sensor to enable realistic perception and manoeuvring. Two algorithms, Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC), are trained and evaluated in randomized scenarios with static obstacles. The framework employs a continuous rudder control action space and a multi-objective reward function with a weighting coefficient to balance path following and obstacle avoidance. Extensive experiments across 1000 test episodes show that SAC consistently outperforms PPO, achieving over 90% success rates, smoother trajectories, and better robustness to varied obstacle layouts. The weighting coefficient is shown to significantly influence agent behaviour, with lambda value of 0.5 providing the best trade-off between safety and efficiency. These results demonstrate the effectiveness of entropy-regularised off-policy methods for ASV navigation and highlight design considerations for future real-world deployment.

Keywords—autonomous surface vessel, reinforcement learning, path planning, collision avoidance

I. INTRODUCTION

Autonomous Surface Vessels (ASVs) are increasingly being deployed in various sectors such as maritime survey, logistic and cargo shipping [1]. Their ability to autonomously navigate open waters in a pre-defined trajectory while avoiding obstacles is crucial for operational safety and efficiency. While traditional methods provide robust solutions, they are often constrained by their inability to adapt to dynamic environments and computational limitations in large-scale settings. To overcome these limitations, machine learning-based techniques have emerged as potential solutions. Deep Reinforcement Learning (DRL) algorithms offer the flexibility of real-time decision-making by training agents to optimize their behaviours in response to environmental inputs [2]. Unlike traditional methods, DRL agents can learn optimal strategies through trial-and-error without relying on explicit environmental models. In this paper, a DRL-based approach is proposed for ASV's path following and static obstacle collision avoidance controls. A comparative study between Proximal Policy Optimization (PPO) and Soft

Actor-Critic (SAC) is performed to demonstrate the differences in efficiency and performance of the two algorithms.

Reinforcement Learning is a learning method where an agent interacts with an environment by receiving observations, taking actions and obtaining reward or penalty, with the objective of maximising long-term cumulative reward [3]. In recent years, it has shown significant potentials in enabling ASVs to autonomously avoid collisions without relying on environmental models [4]. Recent studies have investigated and applied different DRL algorithms to improve path planning and collision avoidance. In [5], Woo and Kim used Deep Q-Networks (DQN) [6] to address the local path planning problem, navigating the ship to avoid nearby obstacles detected by sensor data. Their experimental results showed that the ship could avoid multiple static and dynamic obstacles in complex scenarios. However, training the DRL model in high-dimensional grid map visualisations required extensive training time. Teitgen et al. [7] improved the high-dimensional map problem by implementing a grid-based method for representing the ASV's environment. Their system discretises the observation space into grid cells, where each cell corresponds to a portion of the environment. The collision grid dynamically updates based on the agent's perception of its surroundings, enabling a real-time navigation adjustment. Although this method greatly reduced the computational power, the agent receives unnecessary information, such as the area behind the ship, while it does not retain accurate coordinates of the vessel and the surrounding obstacles.

Li et al. [8] combined DQN with Artificial Potential Field (APF) to handle COLREGs-compliant collision avoidance. The APF improved both the action space and reward formulation, while sensor-based perception is used as input data. Although this hybrid approach improved the decision-making process compared to traditional methods, the authors did not employ a ship kinematics model or environmental dynamics, limiting its applicability to real-world ASVs. Hao et al. [9] proposed a Dynamic and Fast Q-Learning (DFQL) algorithm that accelerates convergence by initialising Q-table with APF, addressing the problem of random exploration in partially known environments. Despite demonstrating improved efficiency compared to classical methods such as Rapidly-

exploring Random Tree (RRT) and APF, this work also relied on a simplified grid-based model without realistic ship dynamics. Hao et al. [10] introduced an improved PPO algorithm designed for vessels navigating on restricted waters, integrating ship domain knowledge and Gated Recurrent Units (GRUs) to capture long-term dependencies. However, the authors focused on dynamic obstacle interactions and complex multi-ship encounters, leaving the application of PPO in static obstacle scenarios less explored.

This paper addresses these research gaps by focusing on static obstacle collision avoidance and path following for ASVs in unpredictable changing environment. A simulated LiDAR sensor is incorporated to emulate realistic perception, and a ship dynamic model is included to better capture the vessel's manoeuvrability. A comparative study is performed between PPO and Soft Actor-Critic (SAC), a widely implemented off-policy algorithm, to evaluate efficiency, stability, and path tracking performance under varied obstacle layouts.

The deep reinforcement learning approach employed in this work is model-free, meaning that the agent learns optimal behaviour through interaction with the environment, without relying on any explicit or learned model of the environment's dynamics [11]. Specifically, PPO and SAC algorithms are utilized to train the control policy. These methods directly optimize the agent's policy based on observed rewards and state transitions, without performing any internal simulation or forward planning. The environment's transition function, which governs how the ASV state evolves in response to control actions, remains unknown to the learning algorithm and is treated as a black box. As such, the policy is updated using sampled trajectories collected from real or simulated interactions, which makes this framework highly suitable for complex, high-dimensional environments like ASV navigation, where modelling dynamics can be challenging or impractical.

PPO is a popular on-policy algorithm that updates the policy using batches of collected trajectories, while ensuring stability in learning. It introduces a clipped objective that limits the magnitude of policy updates to prevent large and unstable changes between iterations [12]. PPO contains two separate neural networks: actor network for the policy, critic network for the value function and optimizes them using stochastic gradient descent. Due to its simplicity and high performance, PPO is a standard choice for both discrete and continuous control problems. However, as an on-policy method, it requires more interaction with the environment to converge compared to off-policy algorithms.

On the other hand, SAC is an off-policy algorithm that optimizes a stochastic policy using a maximum entropy objective [13]. This encourages exploration by maximizing expected reward and policy entropy, promoting more diverse and robust behaviours. SAC employs a more sample-efficient approach by reusing past experience stored in a replay buffer, which results in better data efficiency. It uses two networks, actor and critic, for the policy and Q-value functions, respectively. It also has a temperature parameter that automatically adjust the trade-off between exploration and exploitation. SAC is particularly suited for continuous action spaces and environments with high uncertainty, which aligns

with the ASV's navigation task in cluttered and unpredicted waters.

The contents of the paper are organised as follows. Section II describes the DRL framework, which includes the ship model, agent's observation space, action space and reward function. The experiment setup and results are presented and discussed in Section III. And finally, Section IV concludes the paper with future works.

II. DEEP REINFORCEMENT LEARNING FRAMEWORK

A. Ship Model

This study employs a 3-DOF (Degrees of Freedom) dynamic model to simulate an ASV. The model accounts for surge, sway, and yaw, which are sufficient to capture the essential horizontal plane dynamics of the vessel for path following and collision avoidance tasks.

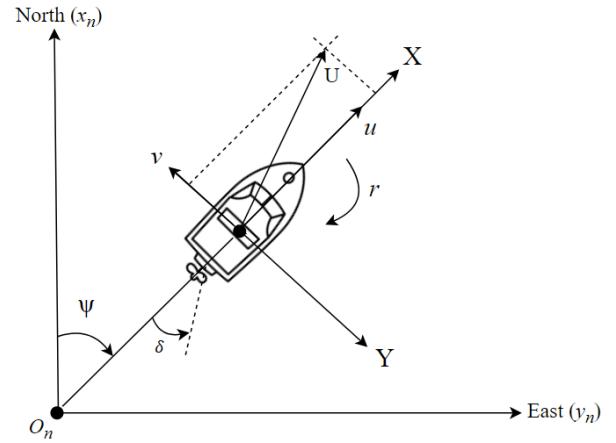


Fig. 1. Earth-fixed NED frame and body-fixed frame of the 3-DOF ship model

The vessel motion is described using two coordinate systems: the Earth-fixed North-East-Down (NED) frame provides an inertial reference, and the body-fixed frame moves with the vessel. The position and orientation of the vessel in the NED frame (global frame) are represented by vector $\eta = [x_n, y_n, \psi]^T$, where x_n and y_n are the Cartesian coordinates and ψ is the heading angle. The vessel's velocity in the body-fixed frame (local frame) is denoted by $v = [u, v, r]^T$, where u is surge velocity, v is sway velocity, and r is yaw rate.

The vessel dynamics are implemented using a force-based modelling approach. The propulsion system generates thrust according to the square of propeller [RPM], while the rudder deflection creates both forward force reduction and turning moment. Additionally, hydrodynamic drag forces act opposite to the direction of motion.

TABLE I. SHIP MODEL SPECIFICATION

Parameter	Name	Value
M	Mass	500 kg
K_T	Thrust coefficient	0.04
K_D	Drag coefficient	10
K_R	Turn coefficient	2800
δ_{max}	Maximum rudder angle	45°
L_R	Rudder offset	3 m
I	Moment of inertia	$0.5 \cdot M \cdot L_R^2$

The dynamic equations governing the vessel's motion can be expressed as:

$$M\dot{v} + D(v)v = \tau \quad (1)$$

where M is the mass matrix
 $D(v)$ is the damping matrix
 τ is the vector of controlling forces and moments

B. Observation Space

For this task, the primary goal is to construct an observation space that provides optimal and meaningful information about the state of the ASV and its surrounding environment from the LiDAR sensor. It is essential to balance between having sufficient parameters for the agent to understand its state and environment. A common approach is using a grid map to provide the observation to the agent [14]. However, the grid-based methods often contain too much unnecessary information and slow down the training process, with insignificant improvements in the agent's performance. Instead, a more efficient approach would be simulating rangefinder beams to "see" the obstacles by calculating the distance of each beam as feedback to the system [15]. To reduce computational resources, the latter approach is implemented.

$$S = [s_p, s_n]^T = [L_{lidar}, (x, y), \psi, \dot{\psi}, d_{path}, \psi_{target}]^T \quad (2)$$

The observation vector S consists of two main features: perception-based s_p and navigation-based s_n as in equation (2). Navigation-based observation lets the agent know the ASV's current state, which includes its coordinates (x, y) , heading angle or yaw ψ , and rate of change of heading angle $\dot{\psi}$. Perception-based navigation represents what the agent observes during navigation, such as surrounding obstacles, desired trajectory and destination. To detect obstacles, a simulated LiDAR sensor is deployed to the front of the ASV. The sensor outputs n -number of light beams, in which each beam has a maximum range of L . The LiDAR beams are evenly distributed across 270 degrees around the ASV. To guide the agent towards following the predefined path, a distance to path d_{path} variable is created by taking the Euclidean distance between the ASV's coordinates and the closest point on the path (Fig. 1). Finally, as the agent is encouraged to head towards the destination, the target heading ψ_{target} is calculated by comparing the relative angle between the ASV's yaw angle to the destination point.

C. Action Space

The ASV is controlled through a continuous action space that modulates the rudder angle δ to influence the vessel's heading ψ . The action space is defined as a continuous variable ranging from -1.0 to 1.0, which is further scaled to $\delta = [-25^\circ, 25^\circ]$. This continuous representation enables smoother control and more realistic compared to the discrete action space. The action is decided at each time step by the RL agent, and the resulting rudder angle is passed to the ship dynamics model that computes and updates the vessel's displacement and heading angle accordingly. To simplify the action space, the vessel is

assumed to travel at a constant speed with the propeller velocity is set to 100 RPM. Therefore, the agent only needs to focus on adjusting the proper rudder angle to navigate the ship through the obstacles.

$$A = [\delta] = [-25^\circ, 25^\circ] = [-1.0, 1.0] \quad (3)$$

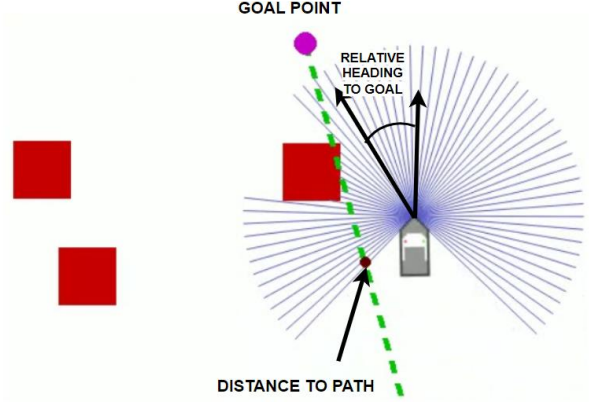


Fig. 2. ASV Observation Space

D. Reward Function

For efficient control and navigation of the ASV, the deep reinforcement learning agent requires a well-designed reward function to balance various objectives: (i) following the desired trajectory, (ii) avoiding obstacles, (iii) maintaining proper heading alignment, (iv) minimising unnecessary movements, (v) reaching the destination. Each component is formulated to provide meaningful feedback to the agent for optimal control while discouraging dangerous or inefficient actions, such as collision with an obstacle or straying off the path.

To discourage excessive movements, a small penalty is applied at every timestep for minimising the total number of steps taken to reach the goal, therefore, improving overall efficiency.

$$r_{exist} = -1 \quad (4)$$

For safe navigation, the ASV is penalised for being too close to the obstacles. Utilising the LiDAR readings from observation space, which provides distance measurements d_i for each LiDAR beam i , a weighted penalty is applied based on the proximity of detected obstacles. The obstacle avoidance reward R_{oa} is presented in equation (5).

$$r_{oa}(t) = -\frac{1}{N} \sum_{i=1}^N \frac{\omega_i}{\max(d_i, 1)} \quad (5)$$

where N is the total of LiDAR beams
 d_i is the distance measured by i -th LiDAR beam
 ω_i is the weight that prioritises beams closer to the forward direction

Equation (6) shows the formula to calculate the weight ω_i by taking account of each LiDAR beam's angle θ_i .

$$\omega_i = \frac{1}{1 + |\theta_i|} \quad (6)$$

When navigating through obstacles, the ASV must follow the predefined path. The deviation from path is measured as the perpendicular distance d_{path} from the ASV's position to the closest point on the path. The path-following reward is formulated using an exponential function:

$$r_{path}(t) = e^{-0.05 |d_{path}|} \quad (7)$$

The exponential function ensures that the agent receives maximum reward when staying directly on the path and is gradually penalised as it strays away. As the agent is encouraged to head towards the goal point, the heading alignment reward $r_{heading}$ is acquired by utilising the target heading parameter ψ_{target} from the observation at every timesteps (equation 8).

$$r_{heading} = \cos \psi_{target} \quad (8)$$

At the end of each episode, the agent reaches terminal state when it reaches the goal point, or collision occurs. Therefore, to strictly discourage the agent from ending the episode early by colliding with an obstacle or the boundary, a large penalty $r_{term} = -1000$ is applied if the training session ends with a collision. Otherwise, a positive reward is given instead, meaning the agent has successfully reached the destination (equation 9). As an extra layer of security, if the distance from the ASV to an obstacle is 10 meters or less, it will be considered as collision to ensure the agent keeps a safe distance from obstacles.

$$r_{term} = \begin{cases} 50 & \text{(if reach goal)} \\ -1000 & \text{(if collision)} \end{cases} \quad (9)$$

Additionally, since the agent needs to accomplish two objectives at the same time: path following and obstacle avoidance, a weighting coefficient $\lambda = [0, 1]$ is added between $r_{path}(t)$ and $r_{oa}(t)$ to balance the cumulative rewards $r(t)$ given to the agent after each episode [16]. The total reward is formulated as:

$$r(t) = \lambda r_{path}(t) + (1 - \lambda)r_{oa}(t) + r_{exist} + r_{heading} + r_{term} \quad (10)$$

Algorithm 1 shows the complete flow of the deep reinforcement learning framework, specifying how the agent interacts with the environment, receiving reward and selecting the next action to update its policy.

Algorithm 1. ASV Environment Interaction Loop

```

Initialize the ASV environment  $\mathbf{E}$  and the control policy  $\pi(\theta)$ 
for episode  $e = 1$  to  $N$ :
    Reset the environment and obtain the initial observation  $s_0$ 
    for time step  $t = 0$  to  $T$ :
        Select rudder action  $a_t$  using the policy  $\pi(\theta)$ 
        Apply  $a_t$  to the ship model to update ASV state
        Receive next state  $s_{t+1}$ , reward  $r_t$ 

```

```

Store the transition  $(s_t, a_t, r_t, s_{t+1})$  in the replay buffer
Record the vessel's path and cumulative reward
If terminate, end the episode and exit the loop
Update the policy

```

III. EXPERIMENT PREPARATION AND RESULTS

A. Training Setup

The agent is trained with PPO and SAC algorithms, using the Python library Stable-Baselines3 [17]. Since PPO and SAC have different training approaches and methods, some agents may have more hyperparameters to tune than others, such as the clip range or entropy coefficient in PPO. Table 2 lists the main hyperparameters and their fine-tuned values prior to the training process. The most notable change in comparison to the default parameters from Stable-Baselines3 is the increase in entropy coefficient from 0.0 to 0.01, which encourages the PPO agent to explore more and reduces exploitation in early stages [18]. In order to evaluate the impact of the weighting coefficient, both PPO and SAC agents are trained with varied λ values, ranging from 0.5 to 0.9, where each training session is run for 1,000,000 episodes.

The training environment is limited to a 400 by 600 meters map, where the boundaries are closed to prevent the ASV from approaching over the edges. At the beginning of each episode, a start point and a goal point are randomly generated along the x-axis with fixed y-coordinates to create a path from the bottom to the top of the map. The purpose of this is to prevent the agent from being biased to a fixed trajectory, which instead tries to actively find and follow the path through exploration. Similarly, for a more dynamic environment, the static obstacles are randomly generated on the map, with the number of obstacles vary from 0 to 10.

TABLE II. HYPERPARAMETERS FOR PPO AND SAC AGENTS

Hyperparameter	Notation	Value
Learning rate	η	0.0001
Number of steps	T	1024
Minibatch size	N_b	512
Discount rate	γ	0.99
Gae lambda	λ	0.95
Clip range	ϵ	0.2
Value function coefficient	c_1	0.5
Entropy coefficient	c_2	0.01

To prepare for evaluation, a set of 1000 episodes with random obstacles, start and destination points were generated to test and compare the performance of PPO and SAC agents with varied weighting coefficient λ , ranging from 0.5 to 0.9.

B. Test Results

a) PPO versus SAC Performance

Table 3 presents the evaluation results including success rate, average cross-track error, standard deviation of cross-track error, and average time elapsed. The success rate is the number of times the agent successfully reaches the destination over 1000 episodes, average cross-track error calculates how far it strays

from the reference path, standard deviation shows the smoothness of the trajectories, and the average time elapsed determines the performance efficiency of each agent.

The statistics indicate that SAC consistently outperformed PPO in terms of success rate, efficiency, trajectory smoothness, and robustness to varying obstacles placements. While prior studies with similar navigation problems often reported PPO as more superior [19], the opposite trend was observed here. This can be explained by several characteristics of both the algorithm and experiment setup. Firstly, SAC is an off-policy method that leverages an experience replay buffer, enabling it to reuse past transitions and achieve greater sample efficiency than PPO [20]. The randomised path and obstacles layouts also favoured SAC's ability to generalise from a broader distribution of samples. Secondly, SAC incorporates entropy regularisation directly into its objective, which encourages exploration and prevents

premature convergence to suboptimal policies [21]. PPO's clipped objective, while effective in stabilising updates, can make the agent biased to exploitation over exploration. Finally, SAC agents produced smoother trajectories because of their Gaussian policy optimisation in continuous action space, whereas PPO requires more careful variance tuning to achieve the same stability. These factors explain why SAC achieved stronger performance than PPO in this experiment.

b) Effect of Weighting Coefficient

The weighting parameter λ , which balances the trade-off between path following and obstacle avoidance tasks, has a decisive impact on the agent's behaviour. Table 3 summarises the statistical evaluation, while Fig. 2-3 illustrate their trajectories across six test scenarios. Overall, $\lambda = 0.5$ and $\lambda = 0.7$ produced the most balanced results with the highest success rate of 95% over 1000 episodes.

TABLE III. EVALUATION OF PPO AND SAC AGENTS ACROSS 1000 EPISODES

Algorithm	λ	Success rate	Avg. cross-track error	Std. cross-track error	Avg. time elapsed
PPO	0.5	56.9%	60.4 m	35.8 m	82.4 s
	0.6	49.4%	59.2 m	34.3 m	80.6 s
	0.7	67.4%	50.9 m	32.4 m	78.5 s
	0.8	50.6%	58.2 m	30.5 m	83.5 s
	0.9	65.3%	54.8 m	27.3 m	80.8 s
SAC	0.5	95.4%	30.2 m	20.2 m	75.8 s
	0.6	90.0%	34.8 m	20.1 m	74.4 s
	0.7	95.5%	47.6 m	29.5 m	79.5 s
	0.8	93.0%	34.1 m	20.3 m	75.0 s
	0.9	91.8%	40.3 m	26.1 m	77.2 s

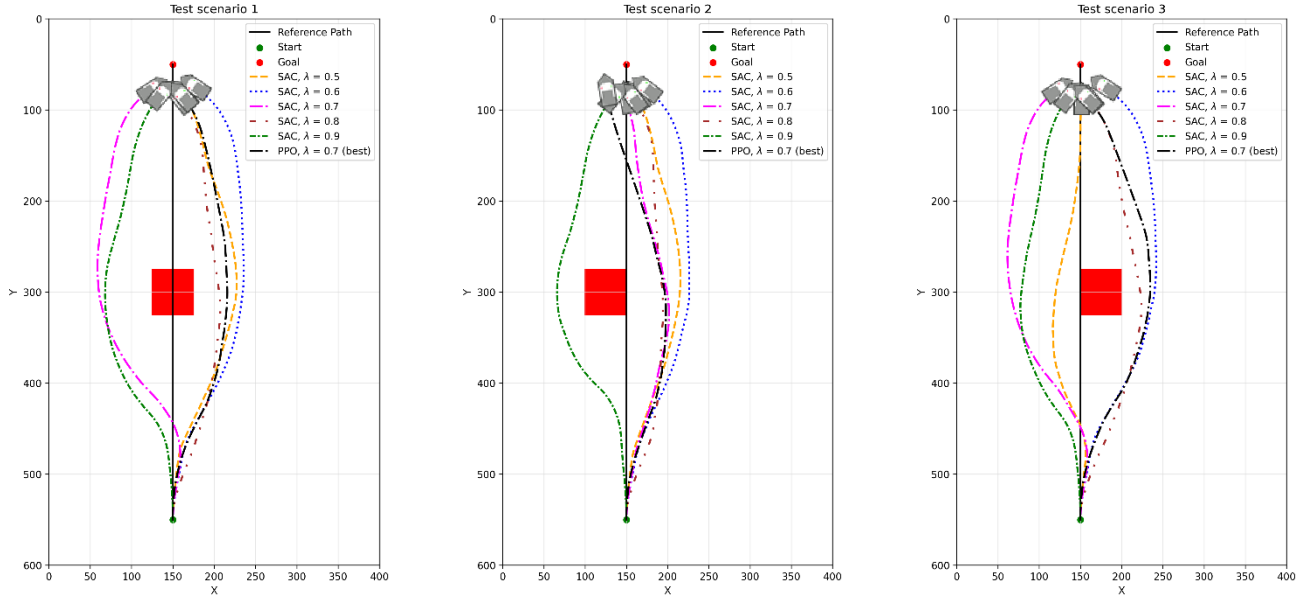


Fig. 3. Comparison of SAC agents with different weighting parameters ($\lambda = 0.5-0.9$) and the best PPO agent in Test Scenarios 1–3 with single obstacles placed at the centre (Scenario 1), slightly left (Scenario 2), and slightly right (Scenario 3) of the reference path.

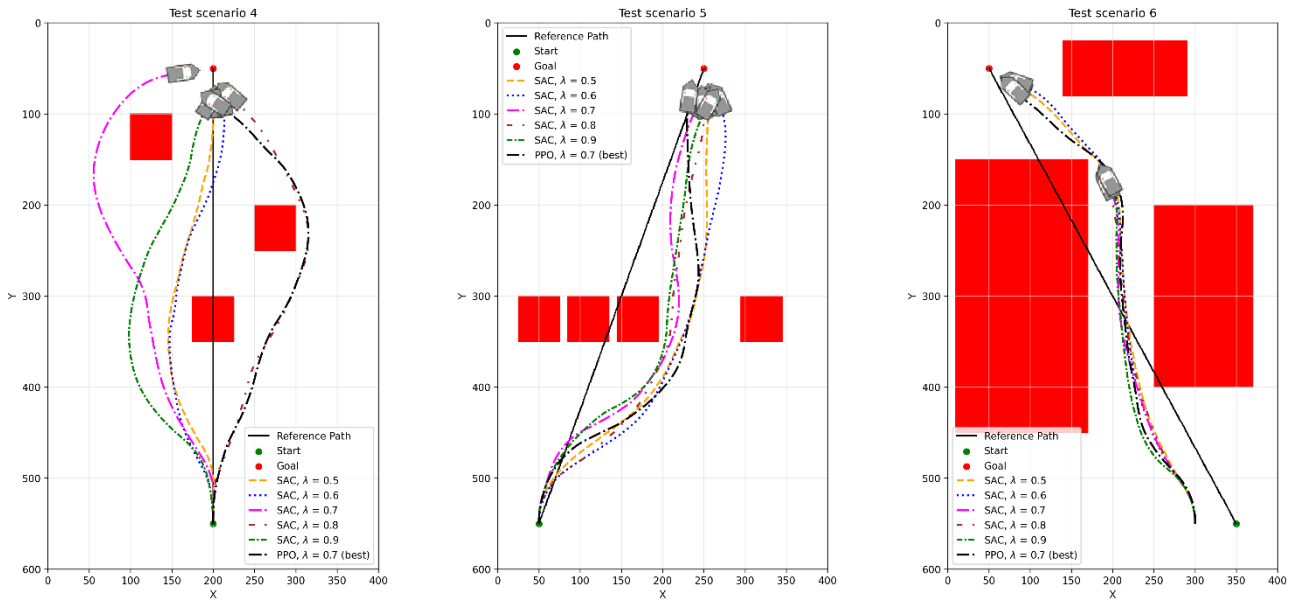


Fig. 4. Comparison of SAC agents and the best PPO agent in Test Scenarios 4–6 with more complex obstacle layouts: obstacles on both sides (Scenario 4), multiple obstacles in a row (Scenario 5), narrow path (Scenario 6).

Furthermore, as shown in the first three test scenarios from Fig. 3, these are the only agents that are not biased to a single strategy when avoiding obstacle, meaning they are more flexible to changes in environment. Fig. 4 presents test cases 4–6, where the $\lambda = 0.5$ agent proved to be more efficient in path following. Most notably in the final test episode, only agents with coefficient λ of 0.5 and 0.6 were able to safely navigate through the narrow path, while others failed to complete the sharp turn. The best PPO agent with $\lambda = 0.7$ is included in Fig. 3–4 to compare against SAC agents. Overall, the PPO agent successfully passed all test cases but with lower efficiency than SAC agents (high cross-track error).

From the statistics and test scenarios analysis, the weighting coefficient $\lambda = 0.5$ provides the most efficient and robust trajectories when navigating through various types of environments. This highlights the importance of tuning reward weights when designing reinforcement learning frameworks.

IV. CONCLUSION AND FUTURE WORKS

This study proposed and evaluated a Deep Reinforcement Learning-based control framework for autonomous surface vessel navigation with static obstacles in unknown environments. The developed framework incorporated LiDAR sensor, randomly generated obstacles, and continuous rudder control to provide a realistic and flexible testbed. Comparative experiments demonstrated that SAC substantially outperformed PPO, achieving higher success rates and smoother trajectories. Among the tested reward weighting parameters, $\lambda = 0.5$ yielded the most effective balance between path following and obstacle collision avoidance, while also exhibiting high efficiency in challenging scenarios, such as narrow seas or dense environments.

Although the results demonstrated strong potential, the current framework is restricted to rudder angle adjustments with fixed speed, which reduces manoeuvrability in constrained passages. Future work will extend the action space to include

both rudder and speed, incorporate dynamic obstacles, and validate the approach on real-world ASV platforms. These directions aim to further enhance the robustness and practical applicability of machine learning in maritime navigation and control.

REFERENCES

- [1] A. Devaraju, L. Chen, and R. R. Negenborn, "Autonomous Surface Vessels in Ports: Applications, Technologies and Port Infrastructures," in *Computational Logistics*, Cham, R. Cerulli, A. Raiconi, and S. Voß, Eds., 2018: Springer International Publishing, pp. 86–105.
- [2] Z. Chu, F. Wang, T. Lei, and C. Luo, "Path Planning Based on Deep Reinforcement Learning for Autonomous Underwater Vehicles Under Ocean Current Disturbance," *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 1, pp. 108–120, 2023, doi: 10.1109/TIV.2022.3153352.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, 2nd ed (Reinforcement learning: An introduction, 2nd ed.). Cambridge, MA, US: The MIT Press, 2018, pp. xxii, 526–xxii, 526.
- [4] Y. Qiao, J. Yin, W. Wang, F. Duarte, J. Yang, and C. Ratti, "Survey of Deep Learning for Autonomous Surface Vehicles in Marine Environments," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 3678–3701, 2023, doi: 10.1109/TITS.2023.3235911.
- [5] J. Woo and N. Kim, "Collision avoidance for an unmanned surface vehicle using deep reinforcement learning," *Ocean Engineering*, vol. 199, p. 107001, 2020, doi: https://doi.org/10.1016/j.oceaneng.2020.107001.
- [6] V. Mnih *et al.*, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [7] R. Teitgen, B. Monsuez, R. Kukla, R. Pasquier, and G. Foinet, "Dynamic trajectory planning for ships in dense environment using collision grid with deep reinforcement learning," *Ocean Engineering*, vol. 281, p. 114807, 2023, doi: https://doi.org/10.1016/j.oceaneng.2023.114807.
- [8] L. Li, D. Wu, Y. Huang, and Z.-M. Yuan, "A path planning strategy unified with a COLREGS collision avoidance function based on deep reinforcement learning and artificial potential field," *Applied Ocean Research*, vol. 113, p. 102759, 2021, doi: https://doi.org/10.1016/j.apor.2021.102759.
- [9] B. Hao, H. Du, and Z. Yan, "A path planning approach for unmanned surface vehicles based on dynamic and fast Q-learning," *Ocean Engineering*, vol. 270, p. 113632, 2023, doi: https://doi.org/10.1016/j.oceaneng.2023.113632.

- [10] S. Hao, W. Guan, Z. Cui, and J. Lu, "USV Collision Avoidance Decision-Making Based on the Improved PPO Algorithm in Restricted Waters," *Journal of Marine Science and Engineering*, vol. 12, no. 8, doi: 10.3390/jmse12081428.
- [11] J. Terven, "Deep Reinforcement Learning: A Chronological Overview and Methods," *AI*, vol. 6, no. 3, doi: 10.3390/ai6030046.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," presented at the Proceedings of the 35th International Conference on Machine Learning, Proceedings of Machine Learning Research, 2018. [Online]. Available: <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- [14] D.-H. Chun, M.-I. Roh, H.-W. Lee, and D. Yu, "Method for collision avoidance based on deep reinforcement learning with path-speed control for an autonomous ship," *International Journal of Naval Architecture and Ocean Engineering*, vol. 16, p. 100579, 2024, doi: <https://doi.org/10.1016/j.ijnaoe.2023.100579>.
- [15] E. Meyer, A. Heiberg, A. Rasheed, and O. San, "COLREG-Compliant Collision Avoidance for Unmanned Surface Vehicle Using Deep Reinforcement Learning," *IEEE Access*, vol. 8, pp. 165344-165364, 2020, doi: 10.1109/ACCESS.2020.3022600.
- [16] E. Meyer, H. Robinson, A. Rasheed, and O. San, "Taming an Autonomous Surface Vehicle for Path Following and Collision Avoidance Using Deep Reinforcement Learning," *IEEE Access*, vol. 8, pp. 41466-41481, 2020, doi: 10.1109/ACCESS.2020.2976586.
- [17] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of machine learning research*, vol. 22, no. 268, pp. 1-8, 2021.
- [18] A. Lixandru, "Proximal Policy Optimization with Adaptive Exploration," *arXiv preprint arXiv:2405.04664*, 2024.
- [19] C. Xu, R. Zhu, and D. Yang, "Karting racing: A revisit to PPO and SAC algorithm," in *2021 International Conference on Computer Information Science and Artificial Intelligence (CISAI)*, 17-19 Sept. 2021, pp. 310-316, doi: 10.1109/CISAI54367.2021.00066.
- [20] D. Arce, J. Solano, and C. Beltrán, "A Comparison Study between Traditional and Deep-Reinforcement-Learning-Based Algorithms for Indoor Autonomous Navigation in Dynamic Scenarios," *Sensors*, vol. 23, no. 24, doi: 10.3390/s23249672.
- [21] Y. Liu, C. Wang, C. Zhao, H. Wu, and Y. Wei, "A Soft Actor-Critic Deep Reinforcement-Learning-Based Robot Navigation Method Using LiDAR," *Remote Sensing*, vol. 16, no. 12, doi: 10.3390/rs1612207.